



TALLER DE PROGRAMACIÓN – CIENCIA DE DATOS

Machine Learning

Es una rama de la IA que permite que una máquina aprenda cosas por sí mismas, sin la necesidad que sea programada específicamente para cada tarea.

Ejemplo:

- a. Los viajes aumentan de precio cuando llueve o hay concierto
 - El modelo analiza en tiempo real la oferta (conductores disponibles), demanda (usuarios solicitando viaje), clima y tráfico.
 - La acción: ajusta el precio al momento para equilibrar el mercado.
- b. Traducción automática (Google Translate)
 - El modelo: ha “leído” casi todo el internet en varios idiomas. Entiende las palabras según el contexto donde se aplique.
 - La acción: genera traducciones fluidas que parecen naturales, no como un “robot”

¿Qué tienen en común? En todos estos casos hay un patrón:

1. **Datos de entrada:** precios, imágenes, textos, etc.
2. **Entrenamiento:** la máquina busca la relación entre esos datos y un resultado.
3. **Predicción:** cuando llega un dato nuevo, la máquina “adivina” que va a suceder o de que se trata.

¿Cómo se entrenan los modelos?

El entrenamiento de un modelo requiere: datos, repetición y corrección. Se necesitan las siguientes etapas:

1. **Recopilación y limpieza de datos:** los datos del mundo real suelen estar “sucios” (fotos borrosas, datos faltantes, etiquetas erróneas). Es necesario limpiar los datos porque en Machine Learning se aplica la regla: Garbage in, Garbage out (si entra basura, sale basura).
2. **Elección del Algoritmo (el cerebro):** dependiendo de lo que se necesite obtener se elige una estructura matemática. Puede ser una **Red Neuronal** (inspirada en el cerebro humano), un **árbol de decisión** (diagrama de flujo complejo) o una **Regresión**.
3. **Proceso de Entrenamiento (iteración):** el proceso realiza estos pasos miles o millones de veces:
 - a. Predicción inicial: al principio no sabe nada, es decir si le entregamos una foto responderá al azar “otra cosa”.
 - b. Cálculo de error: el sistema compara su respuesta con la respuesta correcta. La diferencia entre ambas es el error.

- c. Ajuste: se optimiza a través de la matemática. El algoritmo ajusta sus parámetros internos (llamados “pesos”), para que la próxima vez, al ver una determinada foto, el error sea más pequeño.
- d. Validación y testeo: para saber si la máquina realmente aprendió o solo memorizó las fotos (Overfitting), se utilizan datos que el modelo nunca ha visto. Si el modelo acierta, podemos decir que está listo; si no acierta, es necesario ajustar el entrenamiento.

Fundamentos de modelos

Un modelo es en Machine Learning una representación matemática de un proceso de la vida real. Por ej: un modelo de avión no es un avión real, pero se parece bastante para poder estudiar como vuela en un túnel de viento.

- En machine learning, el modelo es una simplificación de la realidad.
- No puede capturar todos los detalles del universo, pero captura los patrones más importantes para tomar una decisión.

¿Cuál es la diferencia entre algoritmo y modelo? Por ejemplo, considerando preparar un pastel.

- El algoritmo es la receta, los pasos lógicos, es algo genérico.
- Los datos son los ingredientes que se necesitan.
- El modelo es el pastel terminado, el objetivo terminado.

Modelo supervisado (es el modelo con “Profesor”)

Imagina que el modelo es un estudiante que tiene un libro de ejercicios con las respuestas al final.

- Cómo aprende: mira la pregunta, intenta responder, y luego mira la respuesta correcta para corregirse.
- Su objetivo: aprender la relación que existe entre los datos de entrada y la respuesta.
- Resultado: un modelo que sabe predecir o clasificar.
- Ejemplo: identificar si un correo es spam, predecir el valor del dólar, reconocer rostro al desbloquear un teléfono.

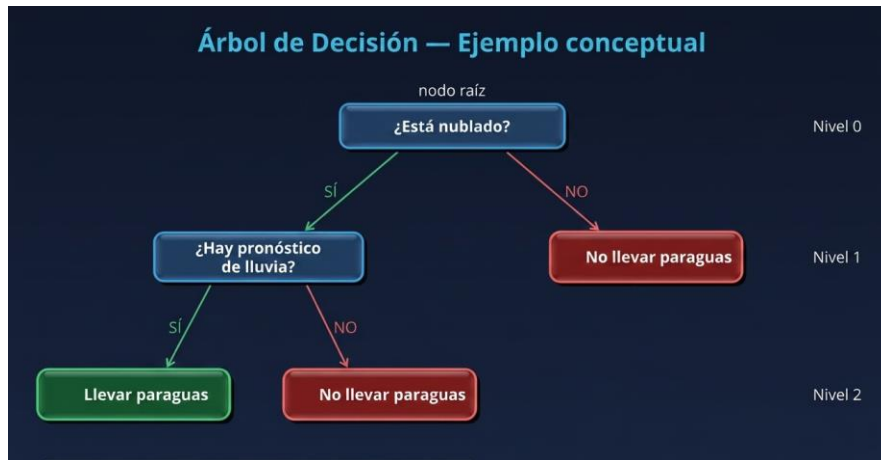
Modelos de Clasificación

Mientras que el modelo de regresión implica predecir un valor continuo (precio, temperatura, ventas), el modelo de clasificación es el algoritmo diseñado para predecir clases discretas (spam/no spam, benigno/maligno). Es el modelo que permite predecir categorías a través de la implementación de su algoritmo.

Árbol de decisión

Un árbol de decisión funciona como un juego de un número determinado de preguntas: realiza preguntas binarias sobre los datos y va descartando hasta llegar a la respuesta correcta.

Ejemplo: ¿llevo paraguas?



Terminología del árbol

- Nodo raíz: corresponde a la primera pregunta.
- Nodo interno: corresponde a una pregunta intermedia.
- Hoja (nodo terminal): corresponde a la decisión final, es decir, la clase que el modelo predice.
- Rama: corresponde al camino que toma (Si/No).
- Profundidad: corresponde a la cantidad de niveles de preguntas que tiene el árbol.

Impureza Gini: responde a la pregunta, ¿cómo elige el árbol la mejor pregunta?

El árbol necesita una métrica para conocer que pregunta divide los datos, y para ello utiliza el concepto de “Impureza”.

Implementación en Scikit-Learn:

Entrenaremos nuestro árbol de decisión en código, visualizamos el árbol resultante, y evaluaremos el modelo con métricas básicas.

Objetivo: predecir si una persona tiene o no riesgo cardíaco (0 – no presenta riesgo, 1 – presenta riesgo).

Descripción del dataset

Contiene variables tales como: edad, presion_arterial, nivel_glucosa, colesterol, indice_masa_corporal, frecuencia_cardiaca, horas_sueno, actividad_fisica, nivel_estres, oxigenacion_sangre, riesgo_cardiaco.

El modelo aprende patrones a partir de las siguientes características médicas como: edad, presión_arterial, glucosa, colesterol, IMC, entre otros parámetros. Por ej: el árbol puede aprender reglas como:

- Si el nivel de glucosa es alto y el colesterol también entonces existe mayor riesgo cardíaco.
- Si la actividad física es alta y el estrés es bajo, el riesgo disminuye.

PASO A PASO:**1. Creamos tres carpetas:**

- data: dataset_medico_normalizado.csv
- src: exploración_dataset.py y árbol_decision.py.
- notebooks: visualización_resultados.ipynb

2. Archivo exploración_dataset.py:

```
exploracion_dataset.py X
src > exploracion_dataset.py > explorar_dataset
1 import pandas as pd
2
3 def cargar_dataset(ruta):
4     df = pd.read_csv(ruta)
5     return df
6
7 def explorar_dataset(df):
8
9     print("=== PRIMEROS REGISTROS ===")
10    print(df.head())
11
12    print("\n=== INFORMACIÓN GENERAL ===")
13    print(df.info())
14
15    print("\n=== COLUMNAS ===")
16    print(df.columns)
17
18    print("\n=== DIMENSIONES ===")
19    print(df.shape)
20
21    print("\n=== ESTADÍSTICAS DESCRIPTIVAS ===")
22    print(df.describe())
23
24    print("\n=== VALORES NULOS ===")
25    print(df.isnull().sum())
```

3. Archivo árbol_decision.py:

```
exploracion_dataset.py  árbol_decision.py X
src > árbol_decision.py > entrenar_modelo
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.metrics import accuracy_score
5 from sklearn.metrics import classification_report
6
7 def entrenar_modelo(df):
8     # variables predictoras
9     X = df.drop("riesgo_cardiaco", axis=1)
10
11     # variable objetivo
12     y = df["riesgo_cardiaco"]
13
14     # dividir datos
15     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
16
17     # crear modelo
18     modelo = DecisionTreeClassifier(criterion="gini", max_depth=3, random_state=42)
19
20     # entrenar
21     modelo.fit(X_train, y_train)
22
23     # predicciones
24     y_pred = modelo.predict(X_test)
25
26     print("=== ACCURACY ===")
27     print(accuracy_score(y_test, y_pred))
28
29     print("\n=== REPORTE DE CLASIFICACIÓN ===")
30     print(classification_report(y_test, y_pred))
31
32     return modelo, X, y
```

Corresponde al entrenamiento del modelo y realiza lo siguiente:

- **X = df.drop("riesgo_cardiaco", axis=1)** : crea la variable X, la cual contiene todas las variables predictoras (edad, colesterol, etc), axis=1 le indica que busque una columna y no una fila.
- **y = df["riesgo_cardiaco"]**: define la variable objetivo, lo que necesitamos predecir.
- **X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3, random_state=42)**: divide los datos en dos grupos, el valor 0.3, le indica que el 30% se guardará para el examen final (X_test, y_test) y el 70% restante lo utilizará para que el modelo estudie (X_train, y_train).
- **modelo = DecisionTreeClassifier(criterion="gini",max_depth=3,random_state=42)**: define el algoritmo que utilizará, y la fórmula matemática que empleará el árbol para decidir (lo realiza por cada columna). El árbol se está limitando a una profundidad de 3 niveles, de esta forma, evitamos que el árbol se vuelva complejo.
- **modelo.fit(X_train, y_train)**: el algoritmo analiza las características de entrenamiento (X_train) junto con los resultados reales (y_train) para descubrir los patrones y reglas lógicas que provocan el riesgo cardíaco.
- **y_pred = modelo.predict(X_test)**: el modelo se enfrenta al examen, pasamos los datos del 30% que guardamos en la prueba (X_test), pero sin entregar las respuestas. El modelo determina las respuestas y las almacena.
- **print(accuracy_score(y_test, y_pred))**: muestra el porcentaje de aciertos globales. Compara las respuestas reales (y_test) con las predicciones del modelo (y_pred). Si el resultado es de 0.85, significa que el modelo acertó en un 85% de los casos.

4. Visualización y preparación de jupyter: revisamos la codificación en el archivo ipynb